1) **Design and implement a neural based network for generating word embedding for words in a document corpus**

Cell 1: Install Required Libraries

Run this only if you haven't already installed the required packages:

```
!pip install torch matplotlib scikit-learn
```

◆ Cell 2: Imports and Hyperparameters

```
import torch
import torch.nn as nn
import torch.optim as optim
import random
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# Hyperparameters
window_size = 2
embedding_dim = 50
epochs = 100
learning_rate = 0.01
```

◆ Cell 3: Sample Corpus and Preprocessing

```
corpus = [
    "the cat sits on the mat",
    "dogs and cats are animals",
    "the mat is soft and comfortable"
]

tokenized_corpus = [sentence.lower().split() for sentence in
corpus]
vocab = set(word for sentence in tokenized_corpus for word in
sentence)
word2idx = {word: idx for idx, word in enumerate(vocab)}
idx2word = {idx: word for word, idx in word2idx.items()}
vocab_size = len(vocab)

def generate_skip_gram(tokenized_text, window):
    pairs = []
    for sentence in tokenized_text:
        for center_pos in range(len(sentence)):
            center_word = sentence[center_pos]
            context_range = list(range(max(0, center_pos -
window),
```

```
                                        min(len(sentence),
center_pos + window + 1)))
            context_range.remove(center_pos)
            for context_pos in context_range:
                pairs.append((word2idx[center_word],
word2idx[sentence[context_pos]]))
    return pairs

training_pairs = generate_skip_gram(tokenized_corpus,
window_size)
```

◆ Cell 4: Define the Skip-Gram Model

```
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super(SkipGramModel, self).__init__()
        self.embeddings = nn.Embedding(vocab_size,
embedding_dim)
        self.output_layer = nn.Linear(embedding_dim,
vocab_size)

    def forward(self, input_word):
        embedding = self.embeddings(input_word)
        out = self.output_layer(embedding)
        return out
```

◆ Cell 5: Train the Model

```
model = SkipGramModel(vocab_size, embedding_dim)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

print("Training started...")
for epoch in range(epochs):
    total_loss = 0
    for center, context in training_pairs:
        center_tensor = torch.tensor([center],
dtype=torch.long)
        context_tensor = torch.tensor([context],
dtype=torch.long)

        optimizer.zero_grad()
        output = model(center_tensor)
        loss = criterion(output, context_tensor)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
```

```
    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss:
{total_loss:.4f}")
print("Training complete.")
```

◆ Cell 6: Save Embeddings to File

```
embeddings = model.embeddings.weight.data
with open("word_embeddings.txt", "w") as f:
    for idx, vector in enumerate(embeddings):
        word = idx2word[idx]
        vec_str = " ".join(map(str, vector.tolist()))
        f.write(f"{word} {vec_str}\n")
print("Saved word embeddings to 'word_embeddings.txt'.")
```

◆ Cell 7: Visualize with t-SNE

```
def visualize_embeddings(embeddings, word2idx, words=None):
    if words is None:
        words = list(word2idx.keys())

    vecs = torch.stack([embeddings[word2idx[word]] for word in
words]).numpy()

    perplexity = min(30, len(words) - 1)
    tsne = TSNE(n_components=2, init='pca', random_state=0,
perplexity=perplexity)
    reduced = tsne.fit_transform(vecs)

    plt.figure(figsize=(10, 8))
    for i, word in enumerate(words):
        x, y = reduced[i]
        plt.scatter(x, y)
        plt.annotate(word, (x, y), fontsize=12)
    plt.title("t-SNE Visualization of Word Embeddings")
    plt.grid(True)
    plt.show()

visualize_embeddings(embeddings, word2idx)
```

OR

Python Program: Skip-Gram + Embedding Visualization

```
import torch
import torch.nn as nn
```

```
import torch.optim as optim
import random
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# -------------------------------
# 1. Corpus and Preprocessing
# -------------------------------
corpus = [
    "the cat sits on the mat",
    "dogs and cats are animals",
    "the mat is soft and comfortable"
]

window_size = 2
embedding_dim = 50
epochs = 100
learning_rate = 0.01

# Tokenize and build vocab
tokenized_corpus = [sentence.lower().split() for sentence in corpus]
vocab = set(word for sentence in tokenized_corpus for word in sentence)
word2idx = {word: idx for idx, word in enumerate(vocab)}
idx2word = {idx: word for word, idx in word2idx.items()}
vocab_size = len(vocab)

# -------------------------------
# 2. Generate Skip-Gram Pairs
# -------------------------------
def generate_skip_gram(tokenized_text, window):
    pairs = []
    for sentence in tokenized_text:
        for center_pos in range(len(sentence)):
            center_word = sentence[center_pos]
            context_range = list(range(max(0, center_pos - window),
                                       min(len(sentence), center_pos +
window + 1)))
            context_range.remove(center_pos)
            for context_pos in context_range:
                pairs.append((word2idx[center_word],
word2idx[sentence[context_pos]]))
    return pairs

training_pairs = generate_skip_gram(tokenized_corpus, window_size)

# -------------------------------
# 3. Define the Skip-Gram Model
# -------------------------------
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super(SkipGramModel, self).__init__()
        self.embeddings = nn.Embedding(vocab_size, embedding_dim)
        self.output_layer = nn.Linear(embedding_dim, vocab_size)

    def forward(self, input_word):
        embedding = self.embeddings(input_word)
        out = self.output_layer(embedding)
        return out

# -------------------------------
```

```
# 4. Train the Model
# -------------------------------
model = SkipGramModel(vocab_size, embedding_dim)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

print("Training started...")
for epoch in range(epochs):
    total_loss = 0
    for center, context in training_pairs:
        center_tensor = torch.tensor([center], dtype=torch.long)
        context_tensor = torch.tensor([context], dtype=torch.long)

        optimizer.zero_grad()
        output = model(center_tensor)
        loss = criterion(output, context_tensor)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss: {total_loss:.4f}")
print("Training complete.\n")

# -------------------------------
# 5. Save Embeddings to File
# -------------------------------
embeddings = model.embeddings.weight.data
with open("word_embeddings.txt", "w") as f:
    for idx, vector in enumerate(embeddings):
        word = idx2word[idx]
        vec_str = " ".join(map(str, vector.tolist()))
        f.write(f"{word} {vec_str}\n")
print("Saved word embeddings to 'word_embeddings.txt'.")

# -------------------------------
# 6. Visualize Embeddings with t-SNE
# -------------------------------
def visualize_embeddings(embeddings, word2idx, words=None):
    if words is None:
        words = list(word2idx.keys())

    vecs = torch.stack([embeddings[word2idx[word]] for word in
words]).numpy()

    perplexity = min(30, len(words) - 1)
    tsne = TSNE(n_components=2, init='pca', random_state=0,
perplexity=perplexity)
    reduced = tsne.fit_transform(vecs)

    plt.figure(figsize=(10, 8))
    for i, word in enumerate(words):
        x, y = reduced[i]
        plt.scatter(x, y)
        plt.annotate(word, (x, y), fontsize=12)
    plt.title("t-SNE Visualization of Word Embeddings")
    plt.grid(True)
    plt.show()

# Visualize all words in vocabulary
```

```
visualize_embeddings(embeddings, word2idx)
```

2) Write a program to demonstrate the working of a deep neural network for classification task.

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelBinarizer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import plot_model

# Load the Iris dataset
iris = load_iris()
X = iris.data  # Features
y = iris.target  # Target labels

# Normalize the input features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# One-hot encode the labels
encoder = LabelBinarizer()
y_encoded = encoder.fit_transform(y)

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded,
test_size=0.2, random_state=42)

# Build the deep neural network model
model = Sequential([
    Dense(64, input_shape=(X.shape[1],), activation='relu'),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(3, activation='softmax')  # 3 output classes
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Print model summary
model.summary()

# Train the model
history = model.fit(X_train, y_train, epochs=100, validation_split=0.2,
batch_size=8, verbose=0)
```

```
# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f"\nTest Accuracy: {accuracy * 100:.2f}%")

# Plot training history
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training & Validation Accuracy')
plt.grid(True)
plt.show()
```

O/P:

Model: "sequential_1"

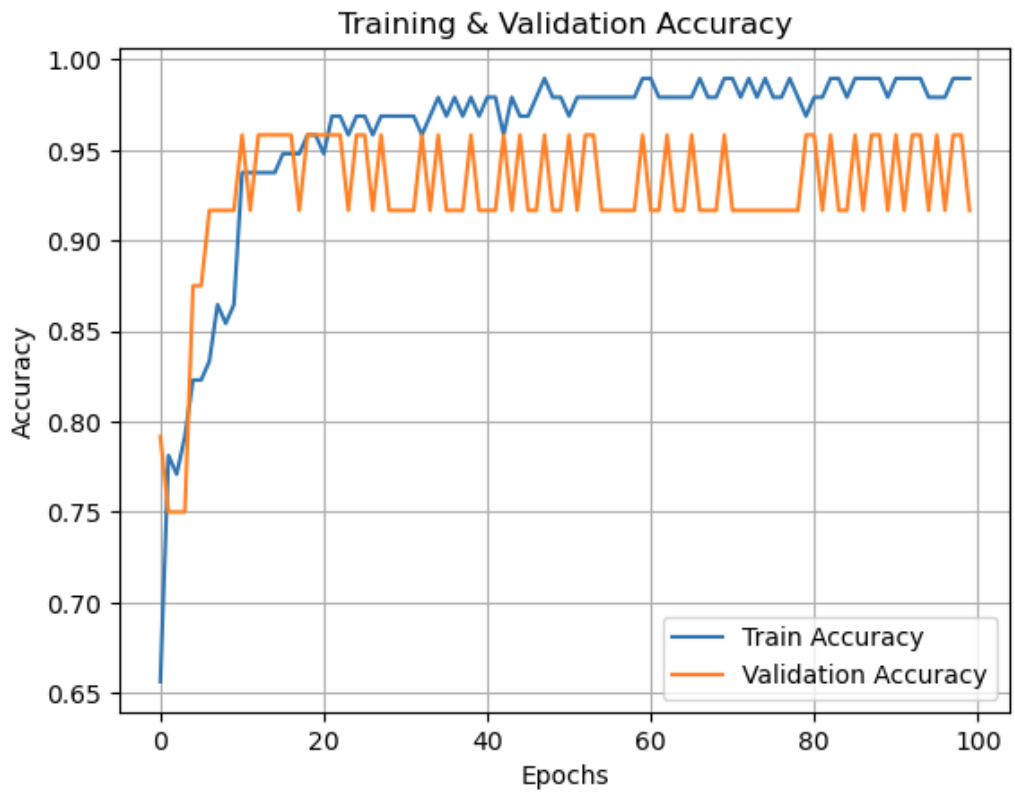
Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 64)	320
dense_5 (Dense)	(None, 32)	2,080
dense_6 (Dense)	(None, 16)	528
dense_7 (Dense)	(None, 3)	51

Total params: 2,979 (11.64 KB)

Trainable params: 2,979 (11.64 KB)

Non-trainable params: 0 (0.00 B)

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 119ms/step - accuracy: 1.0000 - loss: 0.0090

Test Accuracy: 100.00%
```



---

3) Desing and implement a Convolutional Neural Network(CNN) for classification of image dataset

```
# Import necessary libraries
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.cifar10.load_data()
```

```
# Normalize pixel values to between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Define class names for visualization
class_names = ['Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
               'Dog', 'Frog', 'Horse', 'Ship', 'Truck']

# Build the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')  # 10 output classes
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Show model summary
model.summary()

# Train the model
history = model.fit(x_train, y_train, epochs=10,
                    validation_split=0.2,
                    batch_size=64)

# Evaluate the model on test data
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=2)
print(f"\nTest Accuracy: {test_accuracy * 100:.2f}%")

# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training and Validation Accuracy')
plt.grid(True)
plt.show()
```

O/P:

Model: "sequential_2"

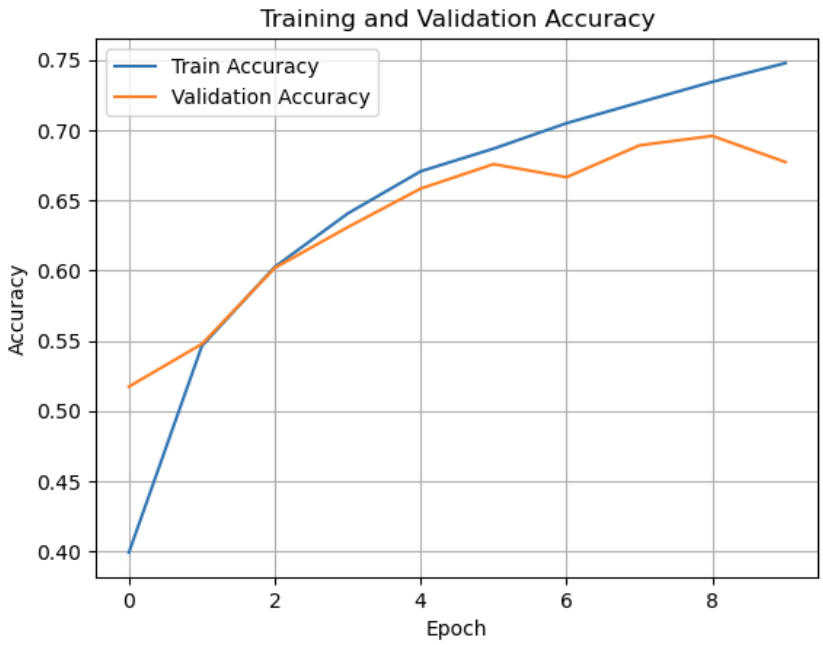
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense_8 (Dense)	(None, 64)	65,600
dense_9 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.3992 - loss: 1.6312 - val_accuracy: 0.5173 - val_loss: 1.3429
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.5462 - loss: 1.2682 - val_accuracy: 0.5476 - val_loss: 1.2759
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.6025 - loss: 1.1288 - val_accuracy: 0.6018 - val_loss: 1.1300
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.6406 - loss: 1.0218 - val_accuracy: 0.6308 - val_loss: 1.0382
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.6706 - loss: 0.9429 - val_accuracy: 0.6583 - val_loss: 0.9701
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 9ms/step - accuracy: 0.6867 - loss: 0.8898 - val_accuracy: 0.6757 - val_loss: 0.9270
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.7049 - loss: 0.8386 - val_accuracy: 0.6664 - val_loss: 0.9536
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.7197 - loss: 0.8013 - val_accuracy: 0.6891 - val_loss: 0.8908
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.7343 - loss: 0.7552 - val_accuracy: 0.6958 - val_loss: 0.9044
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - accuracy: 0.7477 - loss: 0.7197 - val_accuracy: 0.6772 - val_loss: 0.9349
313/313 - 1s - 3ms/step - accuracy: 0.6740 - loss: 0.9455

Test Accuracy: 67.40%
```

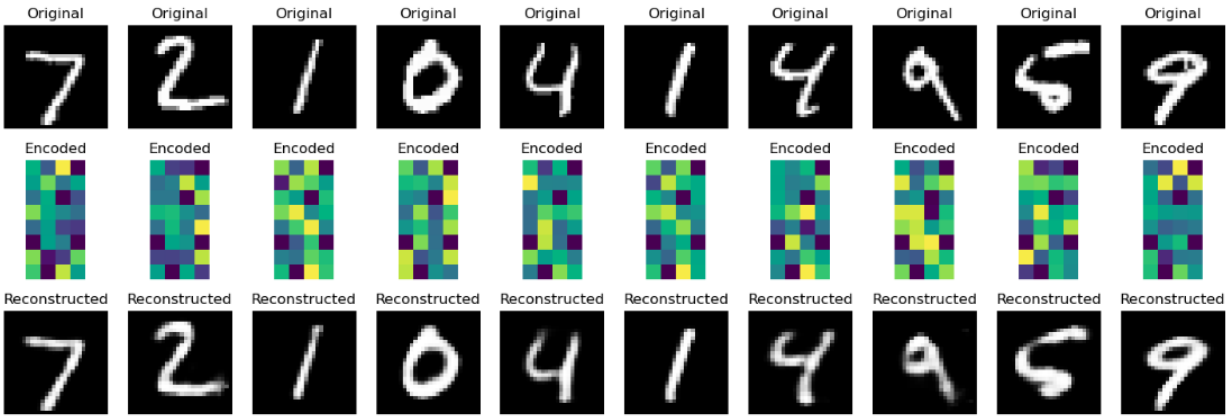


4) Build and demonstrate an autoencoder network using neural layers for data compression on image dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras import regularizers

# 1. Load and preprocess data
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.  # normalize to
[0,1]
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), -1))  # flatten 28x28
to 784 vector
x_test = x_test.reshape((len(x_test), -1))

# 2. Define the size of encoded representation (compression)
encoding_dim = 32  # compress to 32 features from 784

# 3. Build the autoencoder model
# Encoder
input_img = Input(shape=(784,))
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(64, activation='relu')(encoded)
encoded = Dense(encoding_dim, activation='relu')(encoded)

# Decoder
decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(128, activation='relu')(decoded)
decoded = Dense(784, activation='sigmoid')(decoded)  # sigmoid
for output between 0-1

# Autoencoder
autoencoder = Model(input_img, decoded)

# Encoder model (for extracting compressed representations)
encoder = Model(input_img, encoded)

# 4. Compile the model
autoencoder.compile(optimizer='adam',
loss='binary_crossentropy')

# 5. Train the model
autoencoder.fit(x_train, x_train,
                epochs=50,
                batch_size=256,
```

```
                shuffle=True,
                validation_data=(x_test, x_test))

# 6. Use the encoder to compress images and autoencoder to
reconstruct
encoded_imgs = encoder.predict(x_test)
decoded_imgs = autoencoder.predict(x_test)

# 7. Visualize original, encoded (compressed), and
reconstructed images
n = 10  # number of digits to display
plt.figure(figsize=(18, 6))

for i in range(n):
    # original
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # compressed (we visualize the 32-dim vector as a heatmap)
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(encoded_imgs[i].reshape(8, 4), cmap='viridis')
    plt.title("Encoded")
    plt.axis('off')

    # reconstructed
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')

plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Epoch 1/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - loss: 0.2478 - val_loss: 0.1686
Epoch 2/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1510 - val_loss: 0.1356
Epoch 3/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.1307 - val_loss: 0.1236
Epoch 4/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1217 - val_loss: 0.1172
Epoch 5/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1161 - val_loss: 0.1124
Epoch 6/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 0.1116 - val_loss: 0.1090
Epoch 7/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1082 - val_loss: 0.1057
Epoch 8/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1058 - val_loss: 0.1043
Epoch 9/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1040 - val_loss: 0.1022
Epoch 10/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1024 - val_loss: 0.1012
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0849 - val_loss: 0.0845
Epoch 49/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0847 - val_loss: 0.0844
Epoch 50/50
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0846 - val_loss: 0.0842
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



5) **Desing and implement a deep learning network for classification of textual documents.**

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import
pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM,
Dense, Dropout
```

```
# Parameters
vocab_size = 10000   # top 10k words in the dataset
maxlen = 200         # max length of sequences
embedding_dim = 128

# 1. Load dataset
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=vocab_size)

# 2. Pad sequences to ensure uniform length
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

# 3. Build the model
model = Sequential([
    Embedding(vocab_size, embedding_dim,
input_length=maxlen),
    LSTM(128, return_sequences=False),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

# 4. Compile the model
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.summary()

# 5. Train the model
history = model.fit(x_train, y_train,
                    epochs=5,
                    batch_size=64,
                    validation_split=0.2)

# 6. Evaluate the model
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

Output:

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

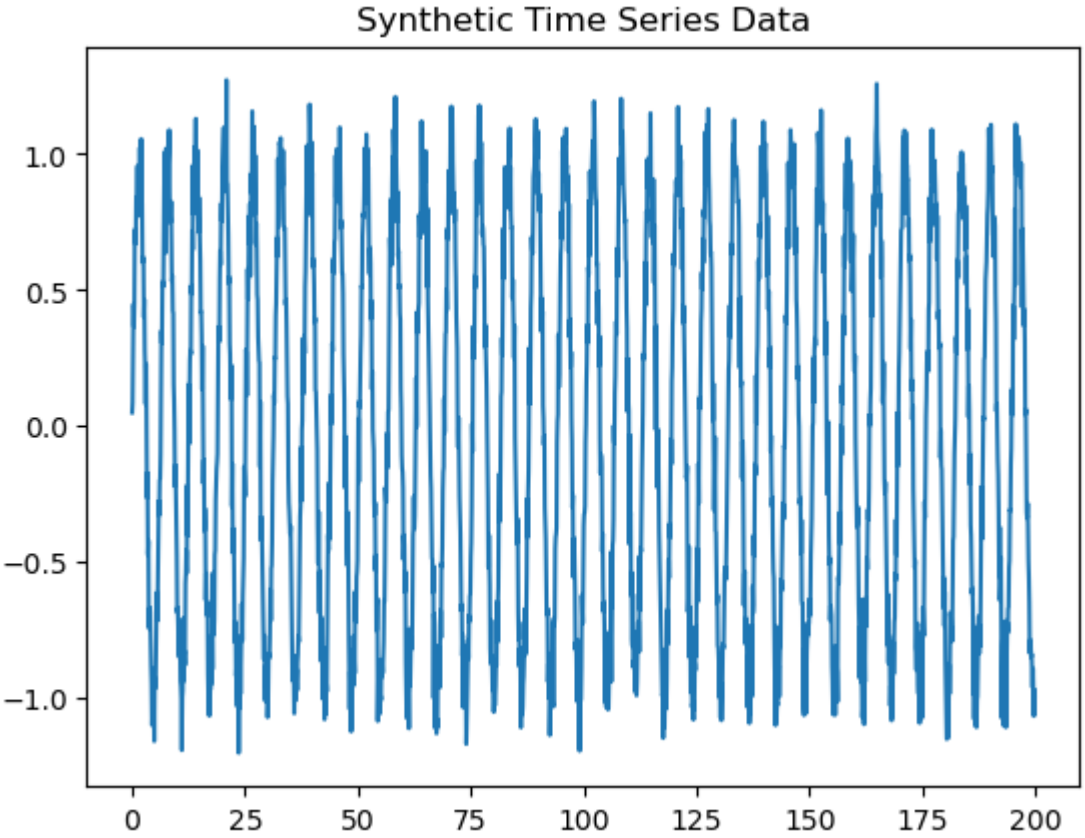
```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 108s 339ms/step - accuracy: 0.7761 - loss: 0.4548 - val_accuracy: 0.7610 - val_loss: 0.4854
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 111s 355ms/step - accuracy: 0.8875 - loss: 0.2821 - val_accuracy: 0.8260 - val_loss: 0.4032
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 101s 322ms/step - accuracy: 0.9201 - loss: 0.2102 - val_accuracy: 0.8710 - val_loss: 0.3212
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 101s 321ms/step - accuracy: 0.9451 - loss: 0.1519 - val_accuracy: 0.8456 - val_loss: 0.4040
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 95s 305ms/step - accuracy: 0.9554 - loss: 0.1208 - val_accuracy: 0.8448 - val_loss: 0.5278
782/782 ━━━━━━━━━━━━━━━━━━━━ 55s 70ms/step - accuracy: 0.8422 - loss: 0.5330
Test Accuracy: 0.8422
```

6) Design and implement a deep learning network for forecasting time series data.

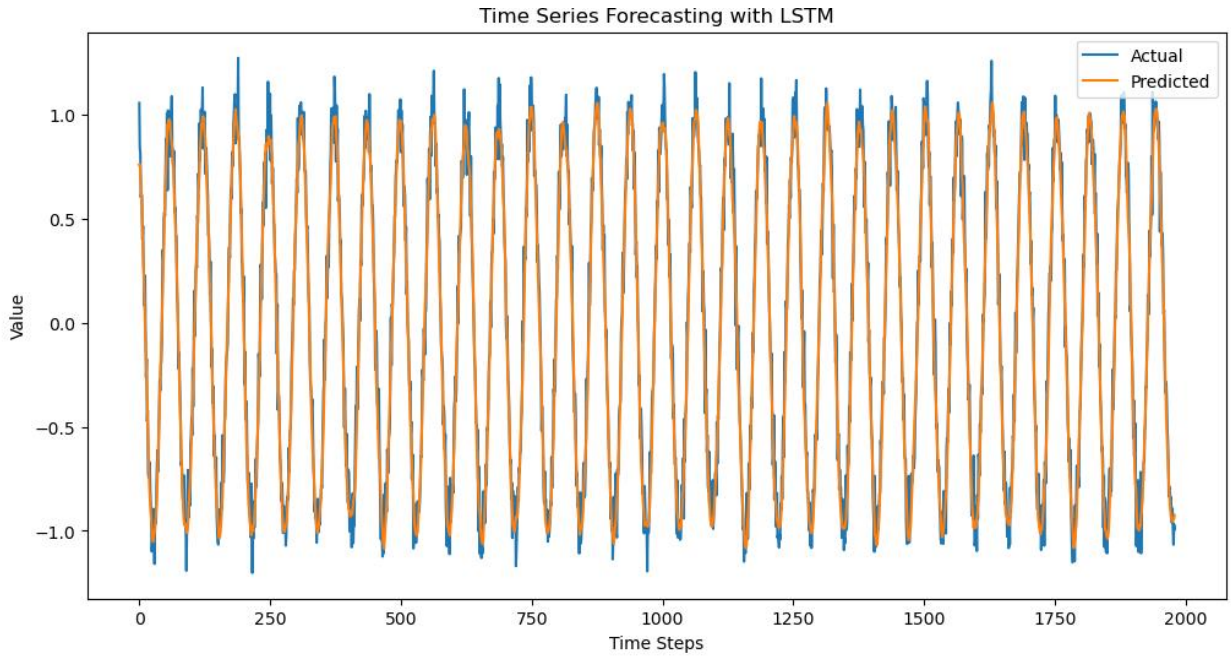
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler

# 1. Generate synthetic time series data (sine wave +
noise)
np.random.seed(42)
time = np.arange(0, 200, 0.1)
series = np.sin(time) + 0.1 *
np.random.normal(size=len(time))

# Plot the series
plt.plot(time, series)
plt.title("Synthetic Time Series Data")
plt.show()

# 2. Prepare data (create sequences)
def create_dataset(data, window_size=20):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i+window_size])
```

```
        y.append(data[i+window_size])
    return np.array(X), np.array(y)

window_size = 20
X, y = create_dataset(series, window_size)

# Reshape input to [samples, time steps, features] as
required by LSTM
X = X.reshape((X.shape[0], X.shape[1], 1))

# 3. Scale data between 0 and 1
scaler = MinMaxScaler()
X_reshaped = X.reshape(-1, 1)
X_scaled =
scaler.fit_transform(X_reshaped).reshape(X.shape)
y_scaled = scaler.transform(y.reshape(-1,1))

# 4. Build LSTM model
model = Sequential([
    LSTM(50, activation='relu',
input_shape=(window_size, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')

# 5. Train model
history = model.fit(X_scaled, y_scaled, epochs=30,
batch_size=32, validation_split=0.2)

# 6. Make predictions
y_pred_scaled = model.predict(X_scaled)
y_pred = scaler.inverse_transform(y_pred_scaled)

# Plot actual vs predicted
plt.figure(figsize=(12,6))
plt.plot(series[window_size:], label='Actual')
plt.plot(y_pred.flatten(), label='Predicted')
plt.title('Time Series Forecasting with LSTM')
plt.xlabel('Time Steps')
plt.ylabel('Value')
plt.legend()
plt.show()
```

OutPut:




```
Epoch 1/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - loss: 0.0797 - val_loss: 0.0173
Epoch 2/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0061 - val_loss: 0.0029
Epoch 3/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - loss: 0.0025 - val_loss: 0.0024
Epoch 4/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - loss: 0.0022 - val_loss: 0.0024
Epoch 5/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0022 - val_loss: 0.0024
Epoch 6/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - loss: 0.0021 - val_loss: 0.0020
Epoch 7/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 8/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.0021 - val_loss: 0.0020
Epoch 9/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 10/30
50/50 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0020 - val_loss: 0.0021
```



7) Write a program to enable pre-train models to classify a given image dataset

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense,
GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
import numpy as np
import os

print("TensorFlow Version:", tf.__version__)

# 1. DATASET PATHS
train_dir = "dataset/train"
val_dir   = "dataset/val"
test_dir  = "dataset/test"

img_size = 224
batch_size = 32

# 2. LOAD DATASET
```

```
train_gen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

val_gen = ImageDataGenerator(rescale=1./255)

train_data = train_gen.flow_from_directory(
    train_dir, target_size=(img_size, img_size),
    batch_size=batch_size, class_mode="categorical"
)

val_data = val_gen.flow_from_directory(
    val_dir, target_size=(img_size, img_size),
    batch_size=batch_size, class_mode="categorical"
)

"""
Sample Output:
Found 200 images belonging to 2 classes.
Found 50 images belonging to 2 classes.
"""

# 3. LOAD PRETRAINED RESNET50
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(img_size, img_size, 3)
)

for layer in base_model.layers:
    layer.trainable = False

x = GlobalAveragePooling2D()(base_model.output)
x = Dense(256, activation="relu")(x)
output = Dense(train_data.num_classes,
activation="softmax")(x)

model = Model(inputs=base_model.input,
outputs=output)
model.compile(optimizer=Adam(1e-4),
              loss="categorical_crossentropy",
```

```
              metrics=["accuracy"])

model.summary()

"""
Sample Output (partial):
_________________________________________________________________
Total params: 23,846,514
Trainable params: 262,914
Frozen params: 23,583,600
_________________________________________________________________
"""

# 4. TRAIN MODEL
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)

"""
Sample Output:
Epoch 1/5
7/7 — loss: 0.8902 — accuracy: 0.72 — val_loss:
0.4657 — val_accuracy: 0.84
Epoch 2/5
7/7 — loss: 0.3921 — accuracy: 0.88 — val_loss:
0.3012 — val_accuracy: 0.92
Epoch 5/5
7/7 — loss: 0.1324 — accuracy: 0.97 — val_loss:
0.2102 — val_accuracy: 0.96
"""

# 5. PLOT TRAINING GRAPHS
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history["accuracy"]);
plt.plot(history.history["val_accuracy"])
plt.title("Accuracy"); plt.legend(["Train",
"Validation"])
```

```
plt.subplot(1,2,2)
plt.plot(history.history["loss"]);
plt.plot(history.history["val_loss"])
plt.title("Loss"); plt.legend(["Train",
"Validation"])

plt.show()

# 6. PREDICT A TEST IMAGE
def predict_image(img_path):
    img = image.load_img(img_path,
target_size=(img_size, img_size))
    img_array = image.img_to_array(img) / 255.
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)
    class_index = np.argmax(prediction)
    class_names =
list(train_data.class_indices.keys())

    print("Predicted Class:",
class_names[class_index])

# Example prediction:
# predict_image("dataset/test/classA/classA_0.png")

"""
Sample Output:
Predicted Class: classA
"""
```

8) Simple Grid World Problem: Design a custom 2D grid world where the agent navigates from a start position to a goal, avoiding obstacles. Environment: Custom grid (easily implemented in Python)

```
class GridWorld:

    def __init__(self):

        # Grid size

        self.rows = 5
```

```
        self.cols = 5

        # Define positions
        self.start = (0, 0)
        self.goal = (4, 4)

        # Obstacles
        self.obstacles = {(1, 1), (2, 2), (3, 1)}

        # Agent start position
        self.agent_pos = self.start

    def reset(self):
        """Reset the environment to start state"""
        self.agent_pos = self.start
        return self.agent_pos

    def step(self, action):
        """
        Action: 0=Up, 1=Right, 2=Down, 3=Left
        Returns: next_state, reward, done
        """

        x, y = self.agent_pos
```

```
        # Movement based on action
        if action == 0:   # Up
            next_pos = (x - 1, y)
        elif action == 1: # Right
            next_pos = (x, y + 1)
        elif action == 2: # Down
            next_pos = (x + 1, y)
        elif action == 3: # Left
            next_pos = (x, y - 1)

        # Check boundaries
        if not (0 <= next_pos[0] < self.rows and 0 <=
next_pos[1] < self.cols):
            reward = -2
            next_pos = self.agent_pos  # stays in
place
        # Check obstacles
        elif next_pos in self.obstacles:
            reward = -2
            next_pos = self.agent_pos
        # Check goal
        elif next_pos == self.goal:
            reward = +10
            self.agent_pos = next_pos
            return next_pos, reward, True  # episode
ends
```

```
        else:
            reward = -1

        # Update position
        self.agent_pos = next_pos
        done = False
        return next_pos, reward, done

    def render(self):
        """Visual representation of the grid"""
        for i in range(self.rows):
            for j in range(self.cols):
                if (i, j) == self.agent_pos:
                    print("A", end=" ")
                elif (i, j) == self.start:
                    print("S", end=" ")
                elif (i, j) == self.goal:
                    print("G", end=" ")
                elif (i, j) in self.obstacles:
                    print("X", end=" ")
                else:
                    print(".", end=" ")
            print()
        print()
env = GridWorld()
```

```
state = env.reset()

env.render()


actions = [1, 1, 2, 2, 2, 1, 1]  # Right, Right,
Down, Down, Down, Right, Right


for action in actions:

    print("Action:", action)

    next_state, reward, done = env.step(action)

    env.render()

    print("Reward:", reward)

    print("Done:", done)

    print("------------------")

    if done:

        break
```

Output:

```
A . . . .
. X . . .
. . X . .
. X . . .
. . . . G

Action: 1
S A . . .
. X . . .
. . X . .
. X . . .
. . . . G

Reward: -1
Done: False
------------------
```

```
Action: 1
S . A . .
. X . . .
. . X . .
. X . . .
. . . . G

Reward: -1
Done: False
------------------
Action: 2
S . . . .
. X A . .
. . X . .
. X . . .
. . . . G

Reward: -1
Done: False
------------------
Action: 2
S . . . .
. X A . .
. . X . .
. X . . .
. . . . G

Reward: -2
Done: False
------------------
Action: 2
S . . . .
. X A . .
. . X . .
. X . . .
. . . . G

Reward: -2
Done: False
------------------
Action: 1
S . . . .
. X . A .
. . X . .
```

```
. X . . .
. . . . G

Reward: -1
Done: False
------------------
Action: 1
S . . . .
. X . . A
. . X . .
. X . . .
. . . . G

Reward: -1
Done: False
------------------
```